# PES University — Department of Computer Science & Engineering

### UE22CS352AA: Software Engineering — Lab 4: Vibe Coding (Checkers)

---

## 1. Executive Summary & Objectives

This report documents the end-to-end prompt engineering, defect diagnosis, and feature implementation for Scenario 21 (Checkers) in UE22CS352AA Lab 4. Using Vibe Coding methodology, all broken mechanics were resolved and advanced checkers rules were implemented in 4 systematic iterations, followed by full unit testing.

## 2. Student & Repository Details

**Student:** Abhiroop | **USN:** PES2UG24CS024 | **Section:** 5A

**Starter Repository:** https://github.com/SETAPESU26/21_checkers

**Target Repository:** https://github.com/Abhiroop-24/Lab_4_Software-eng.git

**Branch:** main | **Deliverables Folder:** Lab-4/

## 3. Vibe Coding Prompt History

### *Iteration 1 — Defect Diagnosis & Task 1 (Capture Correctness)*

**User Prompt:** We are inspecting Scenario 21 (Checkers). When running main.py and performing a capture move from (4,1) to (2,3) jumping an opponent piece at (3,2), the jumped piece remains on the board instead of disappearing. Inspect board.py and game.py, diagnose why the captured piece is retained, and implement the fix so jumped pieces disappear and cannot be captured again.

**Assistant Response:** Defect Root Cause:
In board.py, move_piece(board, start, end) only updates board[end] and resets board[start] to '.', completely omitting removal of the jumped piece at intermediate coordinate ((sr+er)//2, (sc+ec)//2).

Solution Applied:
Updated move_piece() to check if abs(end[0] - start[0]) == 2. When true, calculate intermediate coordinates (mr, mc), retrieve board[mr][mc], set board[mr][mc] = '.', and return the captured piece for downstream feedback.
Verified: Jumped pieces are now removed and can never be captured again.

### *Iteration 2 — Task 2 (Win & No-Move Detection)*

**User Prompt:** Now implement Task 2: Detect when one side has no pieces or no legal moves. The game must cleanly end in either case, declare the winner, and display the reason without crashing or looping infinitely.

**Assistant Response:** Implementation:
1. Added has_pieces(board, player) to rules.py to check whether the active player retains any pieces.
2. Added has_legal_moves(board, player) in rules.py to verify whether any valid simple moves or capture jumps exist for the active player.
3. Added check_game_over() in Checkers game loop. Before each turn, if active player has no pieces or no moves, the game prints a clear termination banner ('Game over: [Reason] Player [Winner] wins!') and returns cleanly.

### *Iteration 3 — Task 3 (Complete Checkers Rules)*

**User Prompt:** Implement Task 3: Complete checkers rules including forced-capture behavior, multi-capture turns, and king movement in both directions. Promotion must occur at the correct back row.

**Assistant Response:** Implementation:
1. King Movement: In rules.py, simple_move() and capture_move() allow pieces with 'K' (RK, BK) to move and capture in all 4 diagonal directions (both forward and backward).
2. Forced Captures: Added get_all_captures(board, player). If any capture exists on the board, non-capture moves are rejected with 'A capture is mandatory when available.'
3. Multi-Capture Sequences: After a capture, get_captures_for_piece() inspects the landing square. If further jumps exist, active_jump_pos is locked to that square, enforcing continued jumping with the same piece.
4. Promotion: check_and_promote() crowns Red pieces at row 0 (RK) and Black pieces at row 7 (BK). Under standard checkers rules, crowning on the back row terminates the jump sequence cleanly.

### *Iteration 4 — Task 4 (Move-Level Feedback & Modular Refactoring)*

**User Prompt:** Implement Task 4: Add concise move/capture/promotion feedback. One accepted move should produce exactly one player-facing action result, even if multiple internal legal-move checks occur. Also refactor process_move() to enable clean headless unit testing.

**Assistant Response:** Implementation:
1. Added process_move(start, end) returning (success, message, turn_ended).
2. Standardized feedback strings:
- Simple move: 'R moved from (5, 0) to (4, 1).'
- Promoted move: 'R moved from (1, 2) to (0, 1) and was promoted to King!'
- Capture move: 'R captured B at (3, 2) and jumped to (2, 3).'
- Multi-capture: 'R captured B at (3, 2) and jumped to (2, 3). Multi-capture available from (2, 3)!'
3. Kept game state strictly in memory with zero external dependencies and zero persistent storage.

## 4. Verification & Test Suite Execution

All 9 required test suites passed successfully (Ran 9 tests in 0.001s, Status: OK). 10-second MP4 demonstration videos for before and after states were generated and archived in Lab-4/videos/.